

Trabajo Práctico N° 4: Unidad 3

Bases de Datos

Datos de Entrega	Detalle
Materia	Bases de Datos
Trabajo Práctico	N° 4 – Unidad 3
Fecha de entrega	14/09/2026
Profesor/a	Sergio Neira

Integrantes del Grupo

N°	Apellido y Nombre
1	Agustín Contardi
2	Enzo Giaquinta
3	Ignacio Sanchez
4	Fernando Torrez
5	Salcedo Ramiro

Parte 1 — Laboratorio: consultas analíticas lentas

1.1 Consigna

1. Elegir al menos dos consultas analíticas de queries.sql (por ejemplo, facturación por categoría y mes, o ranking de usuarios por gasto) y ejecutarlas con EXPLAIN ANALYZE sobre la base masiva, antes de cualquier cambio.
2. Identificar en el plan qué algoritmo de join eligió el optimizador (Nested Loop, Hash Join o Merge Join) para cada combinación de tablas.

3. Pasarle a la IA el plan real y pedirle que proponga reescrituras y/o índices adicionales que lo mejoren, justificando la propuesta en términos del nodo de join concreto que aparece en el plan (no en generalidades).
4. Leer línea por línea cada propuesta antes de aplicarla; rechazar lo que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo, volver a medir y completar la tabla de la sección 1.2.

1.2 Tabla de resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Mejora
SPEC 1	Hash Join 547.94 ms tiempo antes	Creación de idx_detalle_pedido_producto _covering y idx_producto_activo_covering	Parallel Hash Join con Index Only Scan sobre producto 376.50 ms tiempo después	~31.3% más rapido
SPEC 2	Hash Right Anti Join 352.68 ms tiempo antes	Mismos índices estratégicos (covering en detalle_pedido y producto)	Nested Loop Anti Join con Index Only Scan sobre detalle_pedido 396.20 ms tiempo después	(Pasa de hash Anti a index Scan en loops)

Parte 2 — Lectura crítica de planes de join interpretados por IA

Plan de query “Productos que nunca fueron vendidos” (Not In):

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
Escanea toda la tabla, filtra	Si	El plan muestra que leyó 10

activo, obtiene 10 filas y las mete en un hash de 9 kB.		filas y se quedó con las activas. No descartó ninguna, por eso son 10. El hash usó 9 kB.
Producto: escanea toda la tabla (50,000 filas), filtra activo, deja 44,939 activas, descarta 5,061 y lee 516 bloques de disco.	Parcialmente	El plan confirma 44,939 activas y 5,061 descartadas, que suman 50,000. Pero "516 bloques de disco" no es exacto: son bloques leídos por PostgreSQL, no necesariamente lecturas físicas al disco.
Hash Join: une registros de dos tablas y devuelve 44,939 filas.	Si	El join y el total de filas son correctos.
Index Only Scan: por cada producto consulta el índice; si hay coincidencia descarta el producto, si no lo conserva. Heap Fetches: 0.	No	El índice se usa para buscar coincidencias y no va al heap (Heap Fetches: 0). Pero el índice no decide descartar ni conservar: solo devuelve coincidencias. Esa decisión la toma el Nested Loop Anti Join.
Nested Loop Anti Join: implementa el NOT EXISTS; por cada fila ejecuta el index scan; si hay coincidencia descarta, si no conserva. Como todos los productos fueron vendidos, el resultado es 0 filas.	Si	El plan muestra que el Anti Join devuelve 0 filas, por lo que ningún producto quedó sin venta. El index scan muestra 1 fila por loop como promedio, pero eso no significa que cada producto tenga exactamente una venta; el Anti Join corta al primer match.
Sort: ordena por p.nombre; como el resultado es 0 filas, el quicksort de 25 kB es instantáneo.	Si	El plan indica orden por p.nombre y usa quicksort con 25 kB. Como no hay filas, ordenar es trivial.

Parte 3 — Consultas resumen, rankings y subconsultas bajo especificación precisa

3.1 Consulta Resumen y Ranking: Top 10 Clientes por Gasto Total

Spec de la consulta:

Objetivo: Obtener el ranking de los 10 clientes con mayor facturación histórica acumulada.

Tablas: cliente, pedido, detalle_pedido.

Filtros de vigencia: cliente.activo = TRUE.

Columnas de salida: id_cliente, nombre_cliente, cantidad_pedidos, monto_total_gastado (redondeado a 2 decimales).

Orden y corte: ORDER BY monto_total_gastado DESC LIMIT 10.

Versión A (Join con GROUP BY – Generada con IA):

```
SELECT
    c.id_cliente,
    c.nombre AS nombre_cliente,
    COUNT(DISTINCT p.id_pedido) AS cantidad_pedidos,
    ROUND(SUM(dp.cantidad * dp.precio_unitario_facturado)::NUMERIC, 2)
AS monto_total_gastado
FROM cliente c
JOIN pedido p ON c.id_cliente = p.id_cliente
JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido
WHERE c.activo = TRUE
GROUP BY c.id_cliente, c.nombre
ORDER BY monto_total_gastado DESC
LIMIT 10;
```

Versión B (Alternativa propia con CTE / Agregación previa):

```
WITH pedidos_totales AS (  
    SELECT  
        p.id_cliente,  
        p.id_pedido,  
        SUM(dp.cantidad * dp.precio_unitario_facturado) AS  
total_pedido  
    FROM pedido p  
    JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido  
    GROUP BY p.id_cliente, p.id_pedido  
)  
SELECT  
    c.id_cliente,  
    c.nombre AS nombre_cliente,  
    COUNT(pt.id_pedido) AS cantidad_pedidos,  
    ROUND(SUM(pt.total_pedido)::NUMERIC, 2) AS monto_total_gastado  
FROM cliente c  
JOIN pedidos_totales pt ON c.id_cliente = pt.id_cliente  
WHERE c.activo = TRUE  
GROUP BY c.id_cliente, c.nombre  
ORDER BY monto_total_gastado DESC  
LIMIT 10;
```

Verificación de equivalencia : Se ejecutaron ambas consultas con
(Consulta A) EXCEPT (Consulta B)

y

(Consulta B) EXCEPT (Consulta A)

Ambas devolvieron 0 filas, validando que el resultado es idéntico.

3.2 Consulta con Subconsulta: Productos con precio superior al promedio de su categoría

Spec de la consulta:

Objetivo: Detectar productos con precio de lista superior al promedio de su categoría.

Tablas: producto, categoría.

Filtros de vigencia: categoría.activo = TRUE y producto.activo = TRUE.

Columnas de salida: id_producto, nombre_producto, categoría, precio_lista,
precio_promedio_categoria (redondeado a 2 decimales).

Orden: Por nombre de categoría ascendente y precio de lista descendente.

Versión A (Subconsulta correlacionada - Generada con IA):

```
SELECT
  p.id_producto,
  p.nombre AS nombre_producto,
  cat.nombre AS categoria,
  p.precio_lista,
  ROUND((
    SELECT AVG(p2.precio_lista)
    FROM producto p2
    WHERE p2.id_categoria = p.id_categoria
      AND p2.activo = TRUE
  ), 2) AS precio_promedio_categoria
FROM producto p
JOIN categoria cat ON p.id_categoria = cat.id_categoria
WHERE cat.activo = TRUE
  AND p.activo = TRUE
  AND p.precio_lista > (
    SELECT AVG(p3.precio_lista)
    FROM producto p3
    WHERE p3.id_categoria = p.id_categoria
      AND p3.activo = TRUE
  )
ORDER BY cat.nombre ASC, p.precio_lista DESC;
```

Versión B (Alternativa propia con Window Functions / OVER PARTITION):

```
WITH productos_con_promedio AS (
  SELECT
    p.id_producto,
    p.nombre AS nombre_producto,
    cat.nombre AS categoria,
    p.precio_lista,
    ROUND(AVG(p.precio_lista) OVER(PARTITION BY p.id_categoria),
  2) AS precio_promedio_categoria
  FROM producto p
  JOIN categoria cat ON p.id_categoria = cat.id_categoria
  WHERE cat.activo = TRUE
    AND p.activo = TRUE
)
SELECT
  id_producto,
  nombre_producto,
  categoria,
  precio_lista,
  precio_promedio_categoria
FROM productos_con_promedio
```

```
WHERE precio_lista > precio_promedio_categoria  
ORDER BY categoria ASC, precio_lista DESC;
```

Verificación de equivalencia: Al ejecutar la prueba con EXCEPT, la diferencia en ambos sentidos arrojó 0 filas, confirmando que las dos lógicas producen el mismo conjunto de datos.

Parte 4 — Competencia de optimización entre equipos

4.1 Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Equipo 1	Creación de índices B-Tree estándar en llaves foráneas (id_pedido, id_producto).	1,250.40 ms	610.20 ms	2.05x
Equipo 2	Reescritura con subconsulta agregada previa + índices en llaves foráneas.	1,250.40 ms	480.10 ms	2.60x
Equipo 3 (Nuestro Equipo)	Covering Indexes (INCLUDE de columnas agregadas) + Índice parcial (WHERE activo = TRUE).	1,250.40 ms	185.30 ms	6.75x

4.2 Justificación técnica del resultado

- **Cambio de Algoritmo:** El optimizador pasó de un Parallel Hash Join sobre la tabla completa a un **Nested Loop con Index Only Scan**.
-
- **Reducción de I/O:** Con los índices cubrientes y filtros de vigencia (activo = TRUE), se logró Heap Fetches: 0, procesando los datos directamente en memoria desde los bloques del índice.
-

4.3 Propuestas de la IA descartadas (para registrar en la DUIA)

- **Propuesta Descartada 1:** Crear índices B-Tree independientes para cada columna del WHERE y GROUP BY.
 -
 - *Motivo de rechazo:* Obligaba al optimizador a hacer un *Bitmap Index Scan* combinado, lo cual aumentó el overhead de lectura en lugar de acelerarlo.
 -
- **Propuesta Descartada 2:** Desactivar Hash Joins a nivel sesión (SET enable_hashjoin = off).
 -
 - *Motivo de rechazo:* Forzó un Nested Loop sobre Sequential Scans, elevando el tiempo total a más de 3,500 ms.

Herramienta	Para qué se usó	Prompt / Spec (resumen)	Se aceptó / Se descartó - Por qué
OpenCode / Kiro	Análisis de plan de ejecución (Parte 1 y 2)	"Explicá nodo por nodo este plan EXPLAIN ANALYZE de la consulta de productos no vendidos"	Se aceptó parcialmente: Se corrigió la interpretación sobre las lecturas de bloques de disco y el comportamiento del Anti Join.

OpenCode / Kiro	Generación de consulta ranking (Parte 3)	<i>"Generá una consulta SQL del Top 10 clientes por gasto con filtros de cliente.activo = TRUE"</i>	Se aceptó: Generó la Versión A con JOIN + GROUP BY. Se validó su equivalencia con EXCEPT contra la Versión B.
OpenCode / Kiro	Propuestas de índices para la competencia (Parte 4)	<i>"Proponé índices para acelerar la consulta de facturación analítica sobre la base masiva"</i>	Se descartó: Propuso índices simples e independientes que generaron <i>Bitmap Index Scans</i> ineficientes.
OpenCode / Kiro	Optimización por Covering Indexes (Parte 4)	<i>"¿Cómo lograr Index Only Scan evitando acceder al Heap?"</i>	Se aceptó: Propuso el uso de INCLUDE e índices parciales (WHERE activo = TRUE), reduciendo el tiempo de 1,250 ms a 185 ms.